

Reviewer Pack — Noncommutative Tensor Product Rank vs BP_ReadTwice Width

Entry 70aa969f37c5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 04:54:00 UTC

Noncommutative Tensor Product Rank vs BP_ReadTwice Width

Entry ID: 70aa969f37c5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 04:54:00 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Tensor Products) **Field B** (complexity object): Complexity Theory: Branching Program Complexity

Statement:

[‘For every noncommutative tensor product of two finite-dimensional matrices A and B over a field F , the noncommutative tensor product rank $\text{np}(A \otimes B)$ is upper-bounded by $\text{BP_ReadTwice}(W(A \otimes B)) + O(\log n)$, where $W(P)$ is the number of wires in read-twice branching program P .’, ‘For every instance of read-twice BP for IP_2 , $\text{np}(A \otimes B) = \Omega(W(G))$, where A and B are the matrices corresponding to G under a fixed embedding.’]

Rationale (proposer’s reasoning):

[‘The noncommutative tensor product rank captures the complexity of entangling operations in quantum systems. If this rank is related to BP_ReadTwice width, it could provide insights into separating quantum complexity classes.’, ‘Previous work has established connections between quantum complexity and tensor products, but a direct link with branching programs is new. This conjecture aims to bridge this gap.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32932b9894c452c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the noncommutative tensor product rank $\text{np}(A \otimes B)$ exceeds $\text{BP_ReadTwice}(W(G))$ by more than 3 standard deviations from their mean difference across at least 100 seeds, or if any seed produces a metric difference greater than 10, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncommutative tensor product rank' AND 'BP_ReadTwice width' - 'read-twice branching program' AND 'tensor product rank' - 'Noncommutative Geometry' AND 'complexity theory branching programs'

Top relevant hits considered: - [http://arxiv.org/abs/1607.03608v3] On the tensor product of linear sites and Grothendieck categories - [http://arxiv.org/abs/2505.01899v3] On functoriality and the tensor product property in noncommutative tensor-triangular geometry - [http://arxiv.org/abs/1808.10521v2] Efficient quantum tensor product expanders and unitary t-designs via the zigzag product - [http://arxiv.org/abs/1308.2123v1] The Control of Branching Morphogenesis - [http://arxiv.org/abs/0812.1616v1] Program for calculating bounds on the minimum rank of a graph using Sage - [http://arxiv.org/abs/1312.5571v2] Interim recommendations for the evaluation of Higgs production cross sections and branching ratios at the LHC in the Two - [http://arxiv.org/abs/1209.3595v2] Noncommutative complex differential geometry - [http://arxiv.org/abs/1304.4062v2] Superconformal nets and noncommutative geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10 # Start with a small size and increase if needed

    # Generate random matrices A and B over GF(2)
    F = [0, 1]
    A = [[random.choice(F) for _ in range(n)] for _ in range(n)]
    B = [[random.choice(F) for _ in range(n)] for _ in range(n)]

    def tensor_product(A, B):
        n = len(A)
        np_A_tensor_B = [[[A[i][k] * B[j][l] for l in range(n)] for k in range(n)] for j in range(n)]
        return np_A_tensor_B

    np_A_tensor_B = tensor_product(A, B)

    # Construct a read-twice BP for IP_2 corresponding to the tensor product
    def bp_read_twice(np_A_tensor_B):
        n = len(np_A_tensor_B)
```

```

width = 0
for i in range(n):
    for j in range(n):
        for k in range(n):
            if np_A_tensor_B[i][j][k] == 1:
                width += 1
return width

W_G = bp_read_twice(np_A_tensor_B)

# Compute the noncommutative tensor product rank
def noncommutative_tensor_product_rank(np_A_tensor_B):
    n = len(np_A_tensor_B)
    rank = 0
    for i in range(n):
        for j in range(n):
            if any(np_A_tensor_B[i][j][k] == 1 for k in range(n)):
                rank += 1
    return rank

np_rank = noncommutative_tensor_product_rank(np_A_tensor_B)

# Calculate the metric value
metric_value = np_rank - W_G - math.log(n, 2)

# Check if the conjecture holds
conjecture_holds = abs(metric_value) <= 3 * math.sqrt(metric_value**2 / 100) or abs(metric_val

return {
    "metric_name": "np(A ⊗ B) - W(G) - log(n)",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"np_rank={np_rank}, W_G={W_G}, n={n}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")
    else:

```

```
print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd9b63ae.py", line 80, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd9b63ae.py", line 33, in run_trial
    np_A_tensor_B = tensor_product(A, B)
                    ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd9b63ae.py", line 30, in tensor_product
    np_A_tensor_B = [[[A[i][k] * B[j][l] for l in range(n)] for k in range(n)] for j in range(n)]
                    ^
```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15822
2	propose	ollama_remote	glm4:latest	0	0	13426
3	preregistration	ollama_remote	glm4:latest	0	0	10041
4	novelty	ollama_remote	glm4:latest	0	0	8763
5	novelty	ollama_remote	glm4:latest	0	0	9666
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17232
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11332
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10147
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11189
10	judge	ollama_remote	glm4:latest	0	0	12013

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119631 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/70aa969f37c5.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/70aa969f37c5.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/70aa969f37c5.tar.gz (if generated)